CHAPTER

FOUR

EFI SYSTEM TABLE

This section describes the entry point to a UEFI image and the parameters that are passed to that entry point. There are three types of UEFI images that can be loaded and executed by firmware conforming to this specification. These are UEFI applications (*UEFI Applications*), UEFI boot service drivers (*UEFI Drivers*), and UEFI runtime drivers (*UEFI Drivers*). UEFI applications include UEFI OS loaders (*UEFI OS Loaders*). There are no differences in the entry point for these three image types.

4.1 UEFI Image Entry Point

The most significant parameter that is passed to an image is a pointer to the System Table (see definition immediately below), the main entry point for a UEFI Image. The System Table contains pointers to the active console devices, a pointer to the Boot Services Table, a pointer to the Runtime Services Table, and a pointer to the list of system configuration tables such as ACPI, SMBIOS, and the SAL System Table. This section describes the System Table in detail.

4.1.1 EFI_IMAGE_ENTRY_POINT

Summary

This is the main entry point for a UEFI Image. This entry point is the same for UEFI applications and UEFI drivers.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_IMAGE_ENTRY_POINT) (
  IN EFI_HANDLE                  ImageHandle,
  IN EFI_SYSTEM_TABLE            *SystemTable
  );
```

Parameters

ImageHandle
The firmware allocated handle for the UEFI image.

SystemTable
A pointer to the EFI System Table.

Description

This function is the entry point to an EFI image. An EFI image is loaded and relocated in system memory by the EFI Boot Service *EFI_BOOT_SERVICES.LoadImage()* . An EFI image is invoked through the EFI Boot Service *EFI_BOOT_SERVICES.StartImage()*.

The first argument is the image's image handle. The second argument is a pointer to the image's system table. The system table contains the standard output and input handles, plus pointers to the *EFI_BOOT_SERVICES* and *EFI_RUNTIME_SERVICES* tables. The service tables contain the entry points in the firmware for accessing the core EFI system functionality. The handles in the system table are used to obtain basic access to the console. In addition, the System Table contains pointers to other standard tables that a loaded image may use if the associated pointers are initialized to nonzero values. Examples of such tables are ACPI, SMBIOS, SAL System Table, etc.

The *ImageHandle* is a firmware-allocated handle that is used to identify the image on various functions. The handle also supports one or more protocols that the image can use. All images support the *EFI_LOADED_IMAGE_PROTOCOL* and the *EFI_LOADED_IMAGE_DEVICE_PATH_PROTOCOL* that returns the source location of the image, the memory location of the image, the load options for the image, etc. The exact *EFI_LOADED_IMAGE_PROTOCOL* and *EFI_LOADED_IMAGE_DEVICE_PATH_PROTOCOL* structures are defined in *EFI_LOADED_IMAGE_PROTOCOL.Unload()* .

If the UEFI image is a UEFI application that is not a UEFI OS loader, then the application executes and either returns or calls the EFI Boot Services *EFI_BOOT_SERVICES.Exit()* . A UEFI application is always unloaded from memory when it exits, and its return status is returned to the component that started the UEFI application.

If the UEFI image is a UEFI OS Loader, then the UEFI OS Loader executes and either returns, calls the EFI Boot Service *Exit()* , or calls the EFI Boot Service *EFI_BOOT_SERVICES.ExitBootServices()* . If the EFI OS Loader returns or calls *Exit()* , then the load of the OS has failed, and the EFI OS Loader is unloaded from memory and control is returned to the component that attempted to boot the UEFI OS Loader. If *ExitBootServices()* is called, then the UEFI OS Loader has taken control of the platform, and EFI will not regain control of the system until the platform is reset. One method of resetting the platform is through the EFI Runtime Service *ResetSystem()* .

If the UEFI image is a UEFI Driver, then the UEFI driver executes and either returns or calls the Boot Service *Exit()* . If the UEFI driver returns an error, then the driver is unloaded from memory. If the UEFI driver returns *EFI_SUCCESS* , then it stays resident in memory. If the UEFI driver does not follow the UEFI Driver Model, then it performs any required initialization and installs its protocol services before returning. If the driver does follow the UEFI Driver Model, then the entry point is not allowed to touch any device hardware. Instead, the entry point is required to create and install the *EFI Driver Binding Protocol* (*EFI Driver Binding Protocol*) on the *ImageHandle* of the UEFI driver. If this process is completed, then *EFI_SUCCESS* is returned. If the resources are not available to complete the UEFI driver initialization, then *EFI_OUT_OF_RESOURCES* is returned.

Status Codes Returned

EFI_SUCCESS	The driver was initialized.
EFI_OUT_OF_RESOURCES	The request could not be completed due to a lack of resources.

4.2 EFI Table Header

The data type *EFI_TABLE_HEADER* is the data structure that precedes all of the standard EFI table types. It includes a signature that is unique for each table type, a revision of the table that may be updated as extensions are added to the EFI table types, and a 32-bit CRC so a consumer of an EFI table type can validate the contents of the EFI table.

4.2.1 EFI_TABLE_HEADER

Summary

Data structure that precedes all of the standard EFI table types.

Related Definitions

```
typedef struct {
  UINT64      Signature;
  UINT32      Revision;
  UINT32      HeaderSize;
  UINT32      CRC32;
  UINT32      Reserved;
 } EFI_TABLE_HEADER;
```

Parameters

Signature
A 64-bit signature that identifies the type of table that follows. Unique signatures have been generated for the EFI System Table, the EFI Boot Services Table, and the EFI Runtime Services Table.

Revision
The revision of the EFI Specification to which this table conforms. The upper 16 bits of this field contain the major revision value, and the lower 16 bits contain the minor revision value. The minor revision value is a decimal value split into two parts, the "upper decimal" and the "lower decimal". The upper decimal is calculated dividing the minor revision value by 10 using integer division. The lower decimal is calculated by taking the minor revision value modulo 10.

When printed or displayed UEFI spec revision is referred as (Major revision).(Minor revision upper digits).(Minor revision lowest digit) or (Major revision).(Minor revision upper digits) in case Minor revision lowest digit is set to 0. For example:

- Specification revision value ((2<<16) | (10)) would be referred to as 2.1;
- Specification revision value ((2<<16) | (30)) would be referred to as 2.3;
- Specification revision value ((2<<16) | (31)) would be referred to as 2.3.1;
- Specification revision value ((2<<16) | (100)) would be referred to as 2.10;
- Specification revision value ((2<<16) | (101)) would be referred to as 2.10.1;

Note that EFI 1.10 did not follow this convention and was referred to as 1.10, not 1.1.

HeaderSize
The size, in bytes, of the entire table including the EFI_TABLE_HEADER.

CRC32
The 32-bit CRC for the entire table. This value is computed by setting this field to 0, and computing the 32-bit CRC for HeaderSize bytes.

Reserved
Reserved field that must be set to 0.

NOTE: The capabilities found in the EFI system table, runtime table and boot services table may change over time. The first field in each of these tables is an EFI_TABLE_HEADER. This header's Revision field is incremented when new capabilities and functions are added to the functions in the table. When checking for capabilities, code should verify that Revision is greater than or equal to the revision level of the table at the point when the capabilities were added to the UEFI specification.

NOTE: Unless otherwise specified, UEFI uses a standard CCITT32 CRC algorithm with a seed polynomial value of 0x04c11db7 for its CRC calculations.

NOTE: The size of the system table, runtime services table, and boot services table may increase over time. It is very important to always use the HeaderSize field of the EFI_TABLE_HEADER to determine the size of these tables.

4.3 EFI System Table

UEFI uses the EFI System Table, which contains pointers to the runtime and boot services tables. The definition for this table is shown in the following code fragments. Except for the table header, all elements in the service tables are pointers to functions as defined in *Services — Boot Services* and *Services — Runtime Services* . Prior to a call to *EFI_BOOT_SERVICES.ExitBootServices()* , all of the fields of the EFI System Table are valid. After an operating system has taken control of the platform with a call to *ExitBootServices()* , only the *Hdr* , *FirmwareVendor* , *FirmwareRevision* , *RuntimeServices* , *NumberOfTableEntries* , and *ConfigurationTable* fields are valid.

4.3.1 EFI_SYSTEM_TABLE

Summary

Contains pointers to the runtime and boot services tables.

Related Definitions

```
#define EFI_SYSTEM_TABLE_SIGNATURE 0x5453595320494249
#define EFI_2_100_SYSTEM_TABLE_REVISION ((2<<16) | (100))
#define EFI_2_90_SYSTEM_TABLE_REVISION  ((2<<16) | (90))
#define EFI_2_80_SYSTEM_TABLE_REVISION  ((2<<16) | (80))
#define EFI_2_70_SYSTEM_TABLE_REVISION  ((2<<16) | (70))
#define EFI_2_60_SYSTEM_TABLE_REVISION  ((2<<16) | (60))
#define EFI_2_50_SYSTEM_TABLE_REVISION  ((2<<16) | (50))
#define EFI_2_40_SYSTEM_TABLE_REVISION  ((2<<16) | (40))
#define EFI_2_31_SYSTEM_TABLE_REVISION  ((2<<16) | (31))
#define EFI_2_30_SYSTEM_TABLE_REVISION  ((2<<16) | (30))
#define EFI_2_20_SYSTEM_TABLE_REVISION  ((2<<16) | (20))
#define EFI_2_10_SYSTEM_TABLE_REVISION  ((2<<16) | (10))
#define EFI_2_00_SYSTEM_TABLE_REVISION  ((2<<16) | (00))
#define EFI_1_10_SYSTEM_TABLE_REVISION  ((1<<16) | (10))
#define EFI_1_02_SYSTEM_TABLE_REVISION  ((1<<16) | (02))
#define EFI_SPECIFICATION_VERSION       EFI_SYSTEM_TABLE_REVISION
#define EFI_SYSTEM_TABLE_REVISION       EFI_2_100_SYSTEM_TABLE_REVISION

typedef struct {
  EFI_TABLE_HEADER                 Hdr;
  CHAR16                           *FirmwareVendor;
  UINT32                           FirmwareRevision;
  EFI_HANDLE                       ConsoleInHandle;
  EFI_SIMPLE_TEXT_INPUT_PROTOCOL   *ConIn;
```

(continues on next page)

(continued from previous page)

```
  EFI_HANDLE                        ConsoleOutHandle;
  EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL  *ConOut;
  EFI_HANDLE                        StandardErrorHandle;
  EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL  *StdErr;
  EFI_RUNTIME_SERVICES             *RuntimeServices;
  EFI_BOOT_SERVICES                *BootServices;
  UINTN                             NumberOfTableEntries;
  EFI_CONFIGURATION_TABLE          *ConfigurationTable;
} EFI_SYSTEM_TABLE;
```

Parameters

Hdr
The table header for the EFI System Table. This header contains the EFI_SYSTEM_TABLE_SIGNATURE and EFI_SYSTEM_TABLE_REVISION values along with the size of the EFI_SYSTEM_TABLE structure and a 32-bit CRC to verify that the contents of the EFI System Table are valid.

FirmwareVendor
A pointer to a null terminated string that identifies the vendor that produces the system firmware for the platform.

FirmwareRevision
A firmware vendor specific value that identifies the revision of the system firmware for the platform.

ConsoleInHandle
The handle for the active console input device. This handle must support *EFI_SIMPLE_TEXT_INPUT_PROTOCOL* and *EFI_SIMPLE_TEXT_INPUT_EX_PROTOCOL*. If there is no active console, these protocols must still be present.

ConIn
A pointer to the *EFI_SIMPLE_TEXT_INPUT_PROTOCOL* interface that is associated with *ConsoleInHandle*.

ConsoleOutHandle
The handle for the active console output device. This handle must support the *EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL*.

If there is no active console, this protocol must still be present.

ConOut
A pointer to the EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL interface that is associated with *ConsoleOutHandle*.

StandardErrorHandle
The handle for the active standard error console device. This handle must support the EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL. If there is no active console, this protocol must still be present.

StdErr
A pointer to the EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL interface that is associated with *StandardErrorHandle*.

RuntimeServices
A pointer to the *EFI Runtime Services Table*.

BootServices
A pointer to the EFI Boot Services Table. See ref:*efi-boot-services-table_efi_system_table*.

NumberOfTableEntries
The number of system configuration tables in the buffer *ConfigurationTable*.

ConfigurationTable
A pointer to the system configuration tables. The number of entries in the table is *NumberOfTableEntries*.

4.4 EFI Boot Services Table

UEFI uses the EFI Boot Services Table, which contains a table header and pointers to all of the boot services. The definition for this table is shown in the following code fragments. Except for the table header, all elements in the EFI Boot Services Tables are prototypes of function pointers to functions as defined in *Services — Boot Services* . The function pointers in this table are not valid after the operating system has taken control of the platform with a call to *EFI_BOOT_SERVICES.ExitBootServices()* .

4.4.1 EFI_BOOT_SERVICES

Summary

Contains a table header and pointers to all of the boot services.

Related Definitions

```
#define EFI_BOOT_SERVICES_SIGNATURE 0x56524553544f4f42
#define EFI_BOOT_SERVICES_REVISION EFI_SPECIFICATION_VERSION

typedef struct {
  EFI_TABLE_HEADER     Hdr;

  //
  // Task Priority Services
  //
  EFI_RAISE_TPL        RaiseTPL;       // EFI 1.0+
  EFI_RESTORE_TPL      RestoreTPL;     // EFI 1.0+

    //
    // Memory Services
    //
    EFI_ALLOCATE_PAGES   AllocatePages;  // EFI 1.0+
    EFI_FREE_PAGES       FreePages;      // EFI 1.0+
    EFI_GET_MEMORY_MAP   GetMemoryMap;   // EFI 1.0+
    EFI_ALLOCATE_POOL    AllocatePool;   // EFI 1.0+
    EFI_FREE_POOL        FreePool;       // EFI 1.0+

    //
    // Event & Timer Services
    //
    EFI_CREATE_EVENT     CreateEvent;    // EFI 1.0+
    EFI_SET_TIMER        SetTimer;       // EFI 1.0+
    EFI_WAIT_FOR_EVENT   WaitForEvent;   // EFI 1.0+
    EFI_SIGNAL_EVENT     SignalEvent;    // EFI 1.0+
    EFI_CLOSE_EVENT      CloseEvent;     // EFI 1.0+
    EFI_CHECK_EVENT      CheckEvent;     // EFI 1.0+

    //
    // Protocol Handler Services
```

(continues on next page)

(continued from previous page)

```
    //
    EFI_INSTALL_PROTOCOL_INTERFACE     InstallProtocolInterface;           // EFI 1.0+
    EFI_REINSTALL_PROTOCOL_INTERFACE   ReinstallProtocolInterface;         // EFI 1.0+
    EFI_UNINSTALL_PROTOCOL_INTERFACE   UninstallProtocolInterface;         // EFI 1.0+
    EFI_HANDLE_PROTOCOL                HandleProtocol;                     // EFI 1.0+
VOID*   Reserved;    // EFI 1.0+
    EFI_REGISTER_PROTOCOL_NOTIFY       RegisterProtocolNotify;             // EFI  1.0+
    EFI_LOCATE_HANDLE                  LocateHandle;                       // EFI 1.0+
    EFI_LOCATE_DEVICE_PATH             LocateDevicePath;                   // EFI 1.0+
EFI_INSTALL_CONFIGURATION_TABLE        InstallConfigurationTable;          // EFI 1.0+

    //
    // Image Services
    //
    EFI_IMAGE_UNLOAD               LoadImage;        // EFI 1.0+
    EFI_IMAGE_START                StartImage;       // EFI 1.0+
    EFI_EXIT                       Exit;             // EFI 1.0+
    EFI_IMAGE_UNLOAD               UnloadImage;      // EFI 1.0+
    EFI_EXIT_BOOT_SERVICES         ExitBootServices; // EFI 1.0+

    //
    // Miscellaneous Services
    //
    EFI_GET_NEXT_MONOTONIC_COUNT   GetNextMonotonicCount; // EFI 1.0+
    EFI_STALL                      Stall;                 // EFI 1.0+
    EFI_SET_WATCHDOG_TIMER         SetWatchdogTimer;      // EFI 1.0+

    //
    // DriverSupport Services
    //
    EFI_CONNECT_CONTROLLER         ConnectController;     // EFI 1.1
    EFI_DISCONNECT_CONTROLLER      DisconnectController;  // EFI 1.1+

    //
    // Open and Close Protocol Services
    //
    EFI_OPEN_PROTOCOL              OpenProtocol;           // EFI 1.1+
    EFI_CLOSE_PROTOCOL             CloseProtocol;          // EFI 1.1+
EFI_OPEN_PROTOCOL_INFORMATION      OpenProtocolInformation;// EFI 1.1+

    //
    // Library Services
    //
    EFI_PROTOCOLS_PER_HANDLE       ProtocolsPerHandle;     // EFI 1.1+
    EFI_LOCATE_HANDLE_BUFFER       LocateHandleBuffer;     // EFI 1.1+
    EFI_LOCATE_PROTOCOL            LocateProtocol;         // EFI 1.1+
 EFI_UNINSTALL_MULTIPLE_PROTOCOL_INTERFACES  InstallMultipleProtocolInterfaces;    //␣
↪EFI 1.1+
 EFI_UNINSTALL_MULTIPLE_PROTOCOL_INTERFACES UninstallMultipleProtocolInterfaces;   //␣
↪EFI 1.1+*

    //
```

(continues on next page)

(continued from previous page)

```
    // 32-bit CRC Services
    //
    EFI_CALCULATE_CRC32    CalculateCrc32;     // EFI 1.1+

    //
    // Miscellaneous Services
    //
    EFI_COPY_MEM           CopyMem;        // EFI 1.1+
    EFI_SET_MEM            SetMem;         // EFI 1.1+
    EFI_CREATE_EVENT_EX    CreateEventEx;  // UEFI 2.0+
  } EFI_BOOT_SERVICES;
```

Parameters

Hdr
The table header for the EFI Boot Services Table. This header contains the EFI_BOOT_SERVICES_SIGNATURE and EFI_BOOT_SERVICES_REVISION values along with the size of the EFI_BOOT_SERVICES structure and a 32-bit CRC to verify that the contents of the EFI Boot Services Table are valid.

RaiseTPL
Raises the task priority level.

RestoreTPL
Restores/lowers the task priority level.

AllocatePages
Allocates pages of a particular type.

FreePages
Frees allocated pages.

GetMemoryMap
Returns the current boot services memory map and memory map key.

AllocatePool
Allocates a pool of a particular type.

FreePool
Frees allocated pool.

CreateEvent
Creates a general-purpose event structure.

SetTimer
Sets an event to be signaled at a particular time.

WaitForEvent
Stops execution until an event is signaled.

SignalEvent
Signals an event.

CloseEvent
Closes and frees an event structure.

CheckEvent
Checks whether an event is in the signaled state.

InstallProtocolInterface
Installs a protocol interface on a device handle.

ReinstallProtocolInterface
Reinstalls a protocol interface on a device handle.

UninstallProtocolInterface
Removes a protocol interface from a device handle.

HandleProtocol
Queries a handle to determine if it supports a specified protocol.

Reserved
Reserved. Must be NULL.

RegisterProtocolNotify
Registers an event that is to be signaled whenever an interface is installed for a specified protocol.

LocateHandle
Returns an array of handles that support a specified protocol.

LocateDevicePath
Locates all devices on a device path that support a specified protocol and returns the handle to the device that is closest to the path.

InstallConfigurationTable
Adds, updates, or removes a configuration table from the EFI System Table.

LoadImag
Loads an EFI image into memory.

StartImage
Transfers control to a loaded image's entry point.

Exit
Exits the image's entry point.

UnloadImage
Unloads an image.

ExitBootServices
Terminates boot services.

GetNextMonotonicCount
Returns a monotonically increasing count for the platform.

Stall
Stalls the processor.

SetWatchdogTimer
Resets and sets a watchdog timer used during boot services time.

ConnectController
Uses a set of precedence rules to find the best set of drivers to manage a controller.

DisconnectController
Informs a set of drivers to stop managing a controller.

OpenProtocol
Adds elements to the list of agents consuming a protocol interface.

CloseProtocol
Removes elements from the list of agents consuming a protocol interface.

OpenProtocolInformtion
Retrieve the list of agents that are currently consuming a protocol interface.

ProtocolsPerHandle
Retrieves the list of protocols installed on a handle. The return buffer is automatically allocated.

LocateHandleBuffer
Retrieves the list of handles from the handle database that meet the search criteria. The return buffer is automatically allocated.

LocateProtocol
Finds the first handle in the handle database the supports the requested protocol.

InstallMultipleProtocolInterfaces
Installs one or more protocol interfaces onto a handle.

UninstallMultipleProtocolInterfaces
Uninstalls one or more protocol interfaces from a handle.

CalculateCrc32
Computes and returns a 32-bit CRC for a data buffer.

CopyMem
Copies the contents of one buffer to another buffer.

SetMem
Fills a buffer with a specified value.

CreateEventEx
Creates an event structure as part of an event group.

4.5 EFI Runtime Services Table

UEFI uses the EFI Runtime Services Table, which contains a table header and pointers to all of the runtime services. The definition for this table is shown in the following code fragments. Except for the table header, all elements in the EFI Runtime Services Tables are prototypes of function pointers to functions as defined in *Services — Runtime Services* . Unlike the EFI Boot Services Table, this table, and the function pointers it contains are valid after the UEFI OS loader and OS have taken control of the platform with a call to *EFI_BOOT_SERVICES.ExitBootServices()*. If a call to *SetVirtualAddressMap()* is made by the OS, then the function pointers in this table are fixed up to point to the new virtually mapped entry points.

4.5.1 EFI_RUNTIME_SERVICES

Summary

Contains a table header and pointers to all of the runtime services.

Related Definitions

```
#define EFI_RUNTIME_SERVICES_SIGNATURE 0x56524553544e5552
#define EFI_RUNTIME_SERVICES_REVISION EFI_SPECIFICATION_VERSION
typedef struct {
    EFI_TABLE_HEADER                Hdr;

    //
    // Time Services
```

(continues on next page)

(continued from previous page)

```
    //
    EFI_GET_TIME                      GetTime;
    EFI_SET_TIME                      SetTime;
    EFI_GET_WAKEUP_TIME               GetWakeupTime;
    EFI_SET_WAKEUP_TIME               SetWakeupTime;

    //
    // Virtual Memory Services
    //
    EFI_SET_VIRTUAL_ADDRESS_MAP       SetVirtualAddressMap;
  EFI_CONVERT_POINTER                 ConvertPointer;

    //
    // Variable Services
    //
    EFI_GET_VARIABLE                  GetVariable;
    EFI_GET_NEXT_VARIABLE_NAME        GetNextVariableName;
    EFI_SET_VARIABLE                  SetVariable;


    //
    // Miscellaneous Services
    //
    EFI_GET_NEXT_HIGH_MONO_COUNT      GetNextHighMonotonicCount;
    EFI_RESET_SYSTEM                  ResetSystem;

   //
   // UEFI 2.0 Capsule Services
   //
   EFI_UPDATE_CAPSULE                UpdateCapsule;
   EFI_QUERY_CAPSULE_CAPABILITIES    QueryCapsuleCapabilities;


 //
 // Miscellaneous UEFI 2.0 Service
 //
  EFI_QUERY_VARIABLE_INFO           QueryVariableInfo;
} EFI_RUNTIME_SERVICES;
```

Parameters

Hdr
The table header for the EFI Runtime Services Table. This header contains the EFI_RUNTIME_SERVICES_SIGNATURE and EFI_RUNTIME_SERVICES_REVISION values along with the size of the EFI_RUNTIME_SERVICES structure and a 32-bit CRC to verify that the contents of the EFI Runtime Services Table are valid.

GetTime
Returns the current time and date, and the time-keeping capabilities of the platform.

SetTime
Sets the current local time and date information.

GetWakeupTime
Returns the current wakeup alarm clock setting.

SetWakeupTime
Sets the system wakeup alarm clock time.

SetVirtualAddressMap
Used by a UEFI OS loader to convert from physical addressing to virtual addressing.

ConvertPointer
Used by EFI components to convert internal pointers when switching to virtual addressing.

GetVariable
Returns the value of a variable.

GetNextVariableName
Enumerates the current variable names.

SetVariable
Sets the value of a variable.

GetNextHighMonotonicCount
Returns the next high 32 bits of the platform's monotonic counter.

ResetSyste
Resets the entire platform.

UpdateCapsule
Passes capsules to the firmware with both virtual and physical mapping.

QueryCapsuleCapabilities
nReturns if the capsule can be supported via UpdateCapsule() .

QueryVariableInfo
Returns information about the EFI variable store.

4.6 EFI Configuration Table & Properties Table

The EFI Configuration Table is the *ConfigurationTable* field in the EFI System Table. This table contains a set of GUID/pointer pairs. Each element of this table is described by the *EFI_CONFIGURATION_TABLE* structure below. The number of types of configuration tables is expected to grow over time. This is why a GUID is used to identify the configuration table type. The EFI Configuration Table may contain at most once instance of each table type.

4.6.1 EFI_CONFIGURATION_TABLE

Summary

Contains a set of GUID/pointer pairs comprised of *ConfigurationTable* field in the EFI System Table.

Related Definitions

```
typedef struct{
  EFI_GUID           VendorGuid;
  VOID               *VendorTable;
}   EFI_CONFIGURATION_TABLE;
```

Parameters

VendorGuid
The 128-bit GUID value that uniquely identifies the system configuration table.

VendorTable

A pointer to the table associated with VendorGuid. Type of the memory that is used to store the table as well as whether this pointer is a physical address or a virtual address during runtime (whether or not a particular address reported in the table gets fixed up when a call to SetVirtualAddressMap() is made) is determined by the VendorGuid. Unless otherwise specified, memory type of the table buffer is defined by the guidelines set forth in the Calling Conventions section in Chapter 2. It is the responsibility of the specification defining the VendorTable to specify additional memory type requirements (if any) and whether to convert the addresses reported in the table. Any required address conversion is a responsibility of the driver that publishes corresponding configuration table.

4.6.1.1 Industry Standard Configuration Tables

The following list shows the GUIDs for tables defined in some of the industry standards. These industry standards define tables accessed as UEFI Configuration Tables on UEFI-based systems. All the addresses reported in these table entries will be referenced as physical and will not be fixed up when transition from preboot to runtime phase. This list is not exhaustive and does not show GUIDs for all possible UEFI Configuration tables.

```
#define EFI_ACPI_20_TABLE_GUID \
  {0x8868e871,0xe4f1,0x11d3,\
    {0xbc,0x22,0x00,0x80,0xc7,0x3c,0x88,0x81}}

#define ACPI_TABLE_GUID \
  {0xeb9d2d30,0x2d88,0x11d3,\
    {0x9a,0x16,0x00,0x90,0x27,0x3f,0xc1,0x4d}}

#define SAL_SYSTEM_TABLE_GUID \
  {0xeb9d2d32,0x2d88,0x11d3,\
   {0x9a,0x16,0x00,0x90,0x27,0x3f,0xc1,0x4d}}

#define SMBIOS_TABLE_GUID \
  {0xeb9d2d31,0x2d88,0x11d3,\
   {0x9a,0x16,0x00,0x90,0x27,0x3f,0xc1,0x4d}}

#define SMBIOS3_TABLE_GUID \
  {0xf2fd1544, 0x9794, 0x4a2c,\
   {0x99,0x2e,0xe5,0xbb,0xcf,0x20,0xe3,0x94})

#define MPS_TABLE_GUID \
  {0xeb9d2d2f,0x2d88,0x11d3,\
   {0x9a,0x16,0x00,0x90,0x27,0x3f,0xc1,0x4d}}
//
// ACPI 2.0 or newer tables should use EFI_ACPI_TABLE_GUID
//
#define EFI_ACPI_TABLE_GUID \
{0x8868e871,0xe4f1,0x11d3,\
{0xbc,0x22,0x00,0x80,0xc7,0x3c,0x88,0x81}}

#define EFI_ACPI_20_TABLE_GUID EFI_ACPI_TABLE_GUID

#define ACPI_TABLE_GUID \
{0xeb9d2d30,0x2d88,0x11d3,\
{0x9a,0x16,0x00,0x90,0x27,0x3f,0xc1,0x4d}}
```

(continues on next page)

(continued from previous page)

```
#define ACPI_10_TABLE_GUID ACPI_TABLE_GUID*
```

4.6.1.2 JSON Configuration Tables

The following list shows the GUIDs for tables defined for reporting firmware configuration data to EFI Configuration Tables and also for processing JSON payload capsule as defined in Section 23.5. The address reported in the table entry identified by EFI_JSON_CAPSULE_DATA_TABLE_GUID will be referenced as physical and will not be fixed up when transition from preboot to runtime phase. The addresses reported in these table entries identified by EFI_JSON_CONFIG_DATA_TABLE_GUID and EFI_JSON_CAPSULE_RESULT_TABLE_GUID will be referenced as virtual and will be fixed up when transition from preboot to runtime phase.

```
#define EFI_JSON_CONFIG_DATA_TABLE_GUID \
{0x87367f87, 0x1119, 0x41ce, \
{0xaa, 0xec, 0x8b, 0xe0, 0x11, 0x1f, 0x55, 0x8a }}

#define EFI_JSON_CAPSULE_DATA_TABLE_GUID \
{0x35e7a725, 0x8dd2, 0x4cac, \
{ 0x80, 0x11, 0x33, 0xcd, 0xa8, 0x10, 0x90, 0x56 }}

#define EFI_JSON_CAPSULE_RESULT_TABLE_GUID \
{0xdbc461c3, 0xb3de, 0x422a,\
{0xb9, 0xb4, 0x98, 0x86, 0xfd, 0x49, 0xa1, 0xe5 }}
```

4.6.1.3 Devicetree Tables

The following list shows the GUIDs for the Devicetree table (DTB). For more information, see "Links to UEFI-Related Documents" (http://uefi.org/uefi) under the headings "Devicetree Specification". The DTB must be contained in memory of type EfiACPIReclaimMemory. The address reported in this table entry will be referenced as physical and will not be fixed up when transition from preboot to runtime phase. Firmware must have the DTB resident in memory and installed in the EFI system table before executing any UEFI applications or drivers that are not part of the system firmware image. Once the DTB is installed as a configuration table, the system firmware must not make any modification to it or reference any data contained within the DTB.

UEFI applications are permitted to modify or replace the loaded DTB. System firmware must not depend on any data contained within the DTB. If system firmware makes use of a DTB for its own configuration, it should use a separate private copy that is not installed in the EFI System Table or otherwise be exposed to EFI applications.

```
//
// Devicetree table, in Flattened Devicetree Blob (DTB) format
//
#define EFI_DTB_TABLE_GUID \
{0xb1b621d5, 0xf19c, 0x41a5, \
{0x83, 0x0b, 0xd9, 0x15, 0x2c, 0x69, 0xaa, 0xe0}}
```

4.6.2 EFI_RT_PROPERTIES_TABLE

This table should be published by a platform if it no longer supports all EFI runtime services once ExitBootServices() has been called by the OS. Note that this is merely a hint to the OS, which it is free to ignore, and so the platform is still required to provide callable implementations of unsupported runtime services that simply return EFI_UNSUPPORTED.

```
#define EFI_RT_PROPERTIES_TABLE_GUID \\
  { 0xeb66918a, 0x7eef, 0x402a, \\
    { 0x84, 0x2e, 0x93, 0x1d, 0x21, 0xc3, 0x8a, 0xe9 }}

      typedef struct {
        UINT16 Version;
        UINT16 Length;
        UINT32 RuntimeServicesSupported;
    } EFI_RT_PROPERTIES_TABLE;
```

Version
Version of the table, must be 0x1

```
#define EFI_RT_PROPERTIES_TABLE_VERSION 0x1
```

Length
Size in bytes of the entire EFI_RT_PROPERTIES_TABLE, must be 8.

RuntimeServicesSupported
Bitmask of which calls are or are not supported, where a bit set to 1 indicates that the call is supported, and 0 indicates that it is not.

```
#define EFI_RT_SUPPORTED_GET_TIME                           0x0001
#define EFI_RT_SUPPORTED_SET_TIME                           0x0002
#define EFI_RT_SUPPORTED_GET_WAKEUP_TIME                    0x0004
#define EFI_RT_SUPPORTED_SET_WAKEUP_TIME                    0x0008
#define EFI_RT_SUPPORTED_GET_VARIABLE                       0x0010
#define EFI_RT_SUPPORTED_GET_NEXT_VARIABLE_NAME             0x0020
#define EFI_RT_SUPPORTED_SET_VARIABLE                       0x0040
#define EFI_RT_SUPPORTED_SET_VIRTUAL_ADDRESS_MAP            0x0080
#define EFI_RT_SUPPORTED_CONVERT_POINTER                    0x0100
#define EFI_RT_SUPPORTED_GET_NEXT_HIGH_MONOTONIC_COUNT      0x0200
#define EFI_RT_SUPPORTED_RESET_SYSTEM                       0x0400
#define EFI_RT_SUPPORTED_UPDATE_CAPSULE                     0x0800
#define EFI_RT_SUPPORTED_QUERY_CAPSULE_CAPABILITIES         0x1000
#define EFI_RT_SUPPORTED_QUERY_VARIABLE_INFO                0x2000
```

The address reported in the EFI configuration table entry of this type will be referenced as physical and will not be fixed up when transitioning from preboot to runtime phase.

4.6.3 EFI_MEMORY_ATTRIBUTES_TABLE

Summary

When published by the system firmware, the EFI_MEMORY_ATTRIBUTES_TABLE provides additional information about regions within the run-time memory blocks defined in the EFI_MEMORY_DESCRIPTOR entries returned from EFI_BOOT_SERVICES . GetMemoryMap() function. The Memory Attributes Table is currently used to describe memory protections that may be applied to the EFI Runtime code and data by an operating system or hypervisor. Consumers of this table must currently ignore entries containing any values for *Type* except for *EfiRuntimeServicesData* and *EfiRuntimeServicesCode* to ensure compatibility with future uses of this table. The Memory Attributes Table may define multiple entries to describe sub-regions that comprise a single entry returned by GetMemoryMap() however the sub-regions must total to completely describe the larger region and may not cross boundaries between entries reported by GetMemoryMap() . If a run-time region returned in GetMemoryMap() entry is not described within the Memory Attributes Table, this region is assumed to not be compatible with any memory protections.

Only entire EFI_MEMORY_DESCRIPTOR entries as returned by GetMemoryMap() may be passed to SetVirtualAddressMap() .

The address reported in the EFI configuration table entry of this type will be referenced as physical and will not be fixed up when transition from preboot to runtime phase.

Prototype

```
#define EFI_MEMORY_ATTRIBUTES_TABLE_GUID \
  {0xdcfa911d, 0x26eb, 0x469f, \
    {0xa2, 0x20, 0x38, 0xb7, 0xdc, 0x46, 0x12, 0x20}}
```

With the following data structure:

```
/*****************************************
/* EFI_MEMORY_ATTRIBUTES_TABLE
/*****************************************
typedef struct {
  UINT32                        Version ;
  UINT32                        NumberOfEntries ;
  UINT32                        DescriptorSize ;
  UINT32                        Flags ;
  // EFI_MEMORY_DESCRIPTOR      Entry [1] ;
  }  EFI_MEMORY_ATTRIBUTES_TABLE;
```

Version
The version of this table. Present version is 0x00000002

NumberOfEntries
Count of EFI_MEMORY_DESCRIPTOR entries provided. This is typically the total number of PE/COFF sections within all UEFI modules that comprise the UEFI Runtime and all UEFI Data regions (e.g. runtime heap).

Entry
Array of *Entries* of type EFI_MEMORY_DESCRIPTOR.

DescriptorSize
Size of the memory descriptor.

Flags
Flags to provide more information for the memory attributes

```
#define EFI_MEMORY_ATTRIBUTES_FLAGS_RT_FORWARD_CONTROL_FLOW_GUARD  0x1
// BIT0 implies that Runtime code includes the forward control flow guard
//      instruction, such as X86 CET-IBT or ARM BTI.
```

Description

For each array entry, the EFI_MEMORY_DESCRIPTOR . *Attribute* field can inform a runtime agency, such as operating system or hypervisor, as to what class of protection settings can be made in the memory management unit for the memory defined by this entry. The only valid bits for *Attribute* field currently are EFI_MEMORY_RO , EFI_MEMORY_XP , plus EFI_MEMORY_RUNTIME . Irrespective of the memory protections implied by *Attribute* , the EFI_MEMORY_DESCRIPTOR . *Type* field should match the type of the memory in enclosing *SetMemoryMap()* entry. *PhysicalStart* must be aligned as specified in *Calling Conventions* . The list must be sorted by physical start address in ascending order. *VirtualStart* field must be zero and ignored by the OS since it has no purpose for this table. *NumPages* must cover the entire memory region for the protection mapping. Each Descriptor in the EFI_MEMORY_ATTRIBUTES_TABLE with attribute *EFI_MEMORY_RUNTIME* must not overlap any other Descriptor in the EFI_MEMORY_ATTRIBUTES_TABLE with attribute EFI_MEMORY_RUNTIME . Additionally, every memory region described by a Descriptor in *EFI_MEMORY_ATTRIBUTES_TABLE* must be a sub-region of, or equal to, a descriptor in the table produced by GetMemoryMap().

Table 4.1: Usage of Memory Attribute Definitions

	EFI_MEMORY_RO	EFI_MEMORY_XP	EFI _MEMORY_RUNTIME
No memory access protection is possible for Entry	0	0	1
Write-protected Code	1	0	1
Read/Write Data	0	1	1
Read-only Data	1	1	1

4.6.4 EFI_CONFORMANCE_PROFILE_TABLE

Summary

This table allows the platform to advertise its UEFI specification conformance in the form of pre-defined profiles. Each profile is identified by a GUID, with known profiles listed in the Description section below.

The absence of this table shall indicate that the platform implementation is conformant with the UEFI specification requirements, as defined in Section 2.6. This is equivalent to publishing this configuration table with the EFI_CONFORMANCE_PROFILES_UEFI_SPEC_GUID conformance profile.

Prototype

```
#define EFI_CONFORMANCE_PROFILES_TABLE_GUID \
{ 0x36122546, 0xf7e7, 0x4c8f, \
{ 0xbd, 0x9b, 0xeb, 0x85, 0x25, 0xb5, 0x0c, 0x0b }}

typedef struct {
UINT16   Version;
UINT16   NumberOfProfiles;
//EFI_GUID ConformanceProfiles [];
} EFI_CONFORMANCE_PROFILES_TABLE;
```

Version

Version of the table, must be 0x1:

```
#define EFI_CONFORMANCE_PROFILES_TABLE_VERSION 0x1
```

NumberOfProfiles

The number of conformance profiles GUIDs present in ConformanceProfiles.

ConformanceProfiles

An array of conformance profile GUIDs that are supported by this system.

The address reported in the EFI configuration table entry of this type will be referenced as physical and will not be fixed up when transition from preboot to runtime phase.

Description

The following list shows the GUIDs of known conformance profiles. This list is not exhaustive and does not show GUIDs for all possible profiles. Additional profiles can be defined and published in other specifications:

```
#define EFI_CONFORMANCE_PROFILES_UEFI_SPEC_GUID \
{ 0x523c91af, 0xa195, 0x4382, \
{ 0x81, 0x8d, 0x29, 0x5f, 0xe4, 0x00, 0x64, 0x65 }}
```

Conformance profile defined by this specification, as defined in Section 2.6.

4.6.5 Other Configuration Tables

The following list shows additional configuration tables defined in this specification:

- EFI_MEMORY_RANGE_CAPSULE_GUID (Section 8.5.3.3)
- EFI_DEBUG_IMAGE_INFO_TABLE (Section 18.4.3)
- EFI_SYSTEM_RESOURCE_TABLE (Section 23.4)
- EFI_IMAGE_EXECUTION_INFO_TABLE (Section 32.5.3.1)
- User Information Table (Section 36.5)
- HII Database export buffer (Section 33.2.11.1)

4.7 Image Entry Point Examples

The examples in the following sections show how the various table examples are presented in the UEFI environment.

4.7.1 Image Entry Point Examples

The following example shows the image entry point for a UEFI Application. This application makes use of the EFI System Table, the EFI Boot Services Table, and the EFI Runtime Services Table.

```
EFI_SYSTEM_TABLE                *gST;
EFI_BOOT_SERVICES               *gBS;
EFI_RUNTIME_SERVICES            *gRT;

EfiApplicationEntryPoint(
  IN EFI_HANDLE                 ImageHandle,
  IN EFI_SYSTEM_TABLE           *SystemTable
```

(continues on next page)

(continued from previous page)

```
  )

 {
  EFI_STATUS Status;
  EFI_TIME                        *Time;

  gST = SystemTable;
  gBS = gST->BootServices;
  gRT = gST->RuntimeServices;

  //
  // Use EFI System Table to print "Hello World" to the active console output
  // device.
  //
  Status = gST->ConOut->OutputString (gST->ConOut, L"Hello world\n\r"); if (EFI_ERROR␣
↪(Status)) {
   return Status;
  }

  //
  // Use EFI Boot Services Table to allocate a buffer to store the current time
  // and date.
  //
  Status = gBS->AllocatePool (
       EfiBootServicesData,
       sizeof (EFI_TIME),
       (VOID \**)&Time
       );
  if (EFI_ERROR (Status)) {
   return Status;
  }

  //
  // Use the EFI Runtime Services Table to get the current time and date.
  //
  Status = gRT->GetTime (Time, NULL)
  if (EFI_ERROR (Status)) {
   return Status;
  }

  return Status;
}
```

The following example shows the UEFI image entry point for a driver that does not follow the UEFI *Driver Model* . Since this driver returns EFI_SUCCESS , it will stay resident in memory after it exits.

```
EFI_SYSTEM_TABLE         *gST;
EFI_BOOT_SERVICES        *gBS;
EFI_RUNTIME_SERVICES     *gRT;

EfiDriverEntryPoint(
  IN EFI_HANDLE           ImageHandle,
```

(continues on next page)

(continued from previous page)

```
  IN EFI_SYSTEM_TABLE     *SystemTable
  )

{
  gST = SystemTable;
  gBS = gST->BootServices;
  gRT = gST->RuntimeServices;

  //
  // Implement driver initialization here.
  //

   return EFI_SUCCESS;
}
```

The following example shows the UEFI image entry point for a driver that also does not follow the UEFI *Driver Model* . Since this driver returns EFI_DEVICE_ERROR, it will not stay resident in memory after it exits.

```
EFI_SYSTEM_TABLE          *gST;
EFI_BOOT_SERVICES         *gBS;
EFI_RUNTIME_SERVICES      *gRT;

EfiDriverEntryPoint(
  IN EFI_HANDLE           ImageHandle,
  IN EFI_SYSTEM_TABLE     *SystemTable
  )

{
  gST = SystemTable;
  gBS = gST->BootServices;
  gRT = gST->RuntimeServices;

  //
  // Implement driver initialization here.
  //

  return EFI_DEVICE_ERROR;
}
```

4.7.2 UEFI Driver Model Example

The following is an UEFI Driver Model example that shows the driver initialization routine for the ABC device controller that is on the XYZ bus. The EFI_DRIVER_BINDING_PROTOCOL and the function prototypes for AbcSupported(), AbcStart(), and AbcStop() are defined in *EFI Driver Binding Protocol* This function saves the driver's image handle and a pointer to the EFI boot services table in global variables, so the other functions in the same driver can have access to these values. It then creates an instance of the EFI_DRIVER_BINDING_PROTOCOL and installs it onto the driver's image handle.

```
extern EFI_GUID               gEfiLoadedImageProtocolGuid;
extern EFI_GUID               gEfiDriverBindingProtocolGuid;
EFI_BOOT_SERVICES             *gBS;
```

(continues on next page)

(continued from previous page)

```
static EFI_DRIVER_BINDING_PROTOCOL  mAbcDriverBinding = {
  AbcSupported,
  AbcStart,
  AbcStop,
  1,
  NULL,
  NULL
};

AbcEntryPoint(
  IN EFI_HANDLE                ImageHandle,
  IN EFI_SYSTEM_TABLE          *SystemTable
  )

{
  EFI_STATUS Status;

  gBS = SystemTable->BootServices;

  mAbcDriverBinding->ImageHandle      = ImageHandle;
  mAbcDriverBinding->DriverBindingHandle = ImageHandle;

  Status = gBS->InstallMultipleProtocolInterfaces(
           &mAbcDriverBinding->DriverBindingHandle,
           &gEfiDriverBindingProtocolGuid, &mAbcDriverBinding,
            NULL
            );
  return Status;
}
```

4.7.3 UEFI Driver Model Example (Unloadable)

The following is the same UEFI Driver Model example as above, except it also includes the code required to allow the driver to be unloaded through the boot service Unload() *EFI_LOADED_IMAGE_PROTOCOL.Unload()* . Any protocols installed or memory allocated in AbcEntryPoint() must be uninstalled or freed in the AbcUnload().

```
 extern EFI_GUID                        gEfiLoadedImageProtocolGuid;
 extern EFI_GUID                        gEfiDriverBindingProtocolGuid;
 EFI_BOOT_SERVICES                      *gBS;
 static EFI_DRIVER_BINDING_PROTOCOL     mAbcDriverBinding = {
   AbcSupported,
   AbcStart,
   AbcStop,
   1,
   NULL,
   NULL
 };

 EFI_STATUS
 AbcUnload (
   IN EFI_HANDLE                        ImageHandle
```

(continues on next page)

(continued from previous page)

```
  );
AbcEntryPoint(
  IN EFI_HANDLE                         ImageHandle,
  IN EFI_SYSTEM_TABLE                   *SystemTable
  )

{
  EFI_STATUS                            Status;
  EFI_LOADED_IMAGE_PROTOCOL             *LoadedImage;

  gBS = SystemTable->BootServices;

  Status = gBS->OpenProtocol (
           ImageHandle,
           &gEfiLoadedImageProtocolGuid,
           &LoadedImage,
           ImageHandle,
           NULL,
           EFI_OPEN_PROTOCOL_GET_PROTOCOL
           );
  if (EFI_ERROR (Status)) {
    return Status;
}
LoadedImage->Unload = AbcUnload;

mAbcDriverBinding->ImageHandle         = ImageHandle;
mAbcDriverBinding->DriverBindingHandle = ImageHandle;

Status =  gBS->InstallMultipleProtocolInterfaces(
          &mAbcDriverBinding->DriverBindingHandle,
          &gEfiDriverBindingProtocolGuid, &mAbcDriverBinding,
          NULL
          );

  return Status;
}

EFI_STATUS Status
AbcUnload (
  IN EFI_HANDLE ImageHandle
  )

{
EFI_STATUS Status;

Status =  gBS->UninstallMultipleProtocolInterfaces (
          ImageHandle,
          &gEfiDriverBindingProtocolGuid, &mAbcDriverBinding,
          NULL
          );
     return Status;
   }
```

4.7.4 EFI Driver Model Example (Multiple Instances)

The following is the same as the first UEFI *Driver Model* example, except it produces three *EFI Driver Binding Protocol* instances. The first one is installed onto the driver's image handle. The other two are installed onto newly created handles.

```
extern EFI_GUID             gEfiDriverBindingProtocolGuid;
EFI_BOOT_SERVICES           *gBS;

static EFI_DRIVER_BINDING_PROTOCOL   mAbcDriverBindingA = {
  AbcSupportedA,
  AbcStartA,
  AbcStopA,
  1,
  NULL,
  NULL
};

static EFI_DRIVER_BINDING_PROTOCOL   mAbcDriverBindingB = {
  AbcSupportedB,
  AbcStartB,
  AbcStopB,
  1,
  NULL,
  NULL
};

static EFI_DRIVER_BINDING_PROTOCOL   mAbcDriverBindingC = {
  AbcSupportedC,
  AbcStartC,
  AbcStopC,
  1,
  NULL,
  NULL
};

AbcEntryPoint(
  IN EFI_HANDLE ImageHandle,
  IN EFI_SYSTEM_TABLE *SystemTable
  )

{
  EFI_STATUS Status;

  gBS = SystemTable->BootServices;

  //
  // Install mAbcDriverBindingA onto ImageHandle
  //
  mAbcDriverBindingA->ImageHandle            = ImageHandle;
  mAbcDriverBindingA->DriverBindingHandle    = ImageHandle;

  Status =  gBS->InstallMultipleProtocolInterfaces(
```

(continues on next page)

(continued from previous page)

```
            &mAbcDriverBindingA->DriverBindingHandle,
            &gEfiDriverBindingProtocolGuid, &mAbcDriverBindingA,
            NULL
            );
  if (EFI_ERROR (Status)) {
     return Status;
  }

  //
  // Install mAbcDriverBindingB onto a newly created handle
  //
  mAbcDriverBindingB->ImageHandle      = ImageHandle;
  mAbcDriverBindingB->DriverBindingHandle      = NULL;

  Status =  gBS->InstallMultipleProtocolInterfaces(
            &mAbcDriverBindingB->DriverBindingHandle,
            &gEfiDriverBindingProtocolGuid, &mAbcDriverBindingB,
            NULL
            );
  if (EFI_ERROR (Status)) {
     return Status;
  }

  //
  // Install mAbcDriverBindingC onto a newly created handle
  //
  mAbcDriverBindingC->ImageHandle = ImageHandle;
  mAbcDriverBindingC->DriverBindingHandle = NULL;

  Status =  gBS->InstallMultipleProtocolInterfaces(
            &mAbcDriverBindingC->DriverBindingHandle,
            &gEfiDriverBindingProtocolGuid, &mAbcDriverBindingC,
            NULL
            );

  return Status;
  }
```